IF

DOCUMENTATION TECHNIQUE · VERSION 1.1

ImmoFlow AI
Manuel du frontend

Direction artistique, architecture React, installation, variables d'environnement et guide d'utilisation espace par espace.

27 ÉCRANS	5 ESPACES	
24 COMPOSANTS	13 200 LIGNES	0 ERREUR TYPESCRIPT

React 18 · TypeScript · Vite · Tailwind · TanStack Query · Zustand

Sommaire

01 Prise en main 4

- Démarrage en trois commandes 4
- Deux façons de lancer l'interface 4
- Scripts 6
- Stack 7
- Ce que contient l'interface 7
- Documentation 8
- Vérifications effectuées avant livraison 8

02 Direction artistique et architecture 9

- 1. Direction artistique 9
- 2. Socle technique 12
- 3. Organisation du code 12
- 4. Trois décisions structurantes 13
- 5. Parcours d'inscription 15
- 6. Qualité de base 15
- 7. Écriture de l'interface 16

03 Installation 17

- Prérequis 17
- Installation 17
- Lancement 18
- Brancher le backend 18
- Build de production 18
- Dépannage 19

04 Variables d'environnement 20

- Référence 20

Fichiers d'exemple 21

05 Guide d'utilisation 23

1. Site public 23

2. Créer un compte 24

3. Espace Super Admin (/admin) 24

4. Espace Agence (/agence) 25

5. Espace Agent (/agent) 26

6. Espace Propriétaire (/proprietaire) 26

7. Espace Client (/espace-client) 26

8. Assistant immobilier 27

9. Préférences 27

10. En cas de problème 28

PARTIE 1

Prise en main

Démarrer en trois commandes, avec ou sans backend.

Interface React de la plateforme immobilière multi-agences ImmoFlow AI.

Démarrage en trois commandes

```
unzip immoflow-ai-frontend.zip
cd immoflow-ai-frontend
npm install
npm run dev
```

L'application est servie sur **http://localhost:5173**.

> Le fichier `.env` est **fourni dans l'archive**, déjà configuré. Aucune étape de configuration n'est nécessaire pour démarrer.

Deux façons de lancer l'interface

A. Branchée sur le backend (configuration par défaut)

C'est le mode livré. `VITE_DEMO_MODE=false` dans `.env`.

Lancez d'abord l'API :

```
cd immoflow-ai-api
docker compose up -d          # API sur http://localhost:8000
```

Puis le frontend :

```
npm run dev
```

Vite relaie automatiquement les appels `/api/*` vers `http://localhost:8000` (voir `vite.config.ts`). Aucune configuration CORS n'est nécessaire en développement.

Connectez-vous avec le compte créé par le backend au premier démarrage :

IDENTIFIANT	MOT DE PASSE
`admin@immoflow.ai`	`ChangeMe123!`

> Sur une base fraîche, le catalogue est vide : c'est normal. Créez une agence depuis `/inscription`, validez-la avec le compte Super Admin, puis invitez un agent et saisissez vos premiers biens. Le parcours complet est décrit dans `docs/GUIDE.md`.

B. Mode démonstration, sans backend

Pour présenter l'interface depuis n'importe quel poste, sans PostgreSQL ni Docker :

```
# dans .env
VITE_DEMO_MODE=true
```

```
npm run dev
```

Un adaptateur HTTP simulé sert alors une soixantaine de routes depuis un jeu de données réaliste : cinq agences, quatre agents, cinq propriétaires, douze biens avec photos, sept visites, trois ventes, des conversations et un journal d'audit. Le périmètre par rôle y est reproduit à l'identique.

Cinq comptes sont proposés en un clic sur l'écran de connexion (n'importe quel mot de passe est accepté) :

COMPTE	RÔLE	CE QU'IL VOIT
`admin@immoflow.ai`	Super Admin	Toutes les agences de la plateforme
`agence@immoflow.ai`	Agence	Son équipe, son portefeuille, ses transactions
`agent@immoflow.ai`	Agent	Ses biens, ses clients, ses visites
`proprietaire@immoflow.ai`	Propriétaire	Les biens qu'il a confiés
`client@immoflow.ai`	Client	Le catalogue publié, ses favoris, ses visites

Scripts

COMMANDE	EFFET
`npm run dev`	Serveur de développement avec rechargement à chaud
`npm run build`	Compilation TypeScript puis build de production dans `dist/`
`npm run preview`	Sert le build de production localement
`npm run typecheck`	Vérification des types sans émission

Stack

DOMAINE	CHOIX
Framework	React 18 + TypeScript strict
Build	Vite 6
Styles	Tailwind CSS 3
Routage	React Router 6
Données serveur	TanStack Query 5
État client	Zustand 5
Formulaires	React Hook Form + Zod
HTTP	Axios
Animation	Framer Motion 11
Icônes	Lucide React
Graphiques	Recharts

Ce que contient l'interface

Pages publiques — accueil avec recherche et statistiques animées, catalogue filtrable, fiche de bien avec galerie plein écran, repère cartographique et prise de rendez-vous.

Authentification — connexion, inscription (client ou agence), mot de passe oublié, réinitialisation. Validation des formulaires par Zod, messages d'erreur en français.

Cinq espaces connectés — Super Admin, Agence, Agent, Propriétaire, Client. Chacun avec son tableau de bord, ses modules et son périmètre de données.

Assistant immobilier — bouton flottant présent partout (`Ctrl` / `Cmd` + `K`), avec trois modes : rechercher un bien en langage courant, estimer un prix, rédiger une annonce. Une page dédiée présente en plus le contrôle qualité des annonces publiées.

Thème clair et sombre, responsive de 360 px au grand écran, focus clavier visible, `prefers-reduced-motion` respecté.

Documentation

DOCUMENT	CONTENU
`docs/ARCHITECTURE.md`	Organisation du code et décisions structurantes
`docs/INSTALLATION.md`	Installation détaillée et dépannage
`docs/ENVIRONNEMENT.md`	Toutes les variables d'environnement
`docs/GUIDE.md`	Guide d'utilisation, espace par espace

Vérifications effectuées avant livraison

- `npm install` puis `npm run build` : **compilation réussie**, 0 erreur TypeScript
- `npm run dev` : serveur démarré, proxy `/api` fonctionnel
- **Appels du frontend rejoués contre l'API réelle** (PostgreSQL 16 + FastAPI) : authentification, rotation de jeton, validation d'agence, cycle complet d'une invitation (création, correction, régénération, révocation de l'ancien lien, activation), les deux parcours d'inscription propriétaire, création et publication de bien, CRM, visites, ventes, messagerie, fonctions d'assistance et isolation inter-agences.
- **Cloisonnement mesuré** sur un jeu de 30 biens : Super Admin 30, Agence 15, Agent 4, Propriétaire 3, Client 20 — le même écran, des chiffres différents.

PARTIE 2

Direction artistique et architecture

Palette, typographie, élément de signature et décisions structurantes.

Ce chapitre documente le frontend React de la plateforme : l'identité visuelle retenue, les conventions de code et l'organisation des écrans.

> **Périmètre livré.** Socle technique, librairie de composants, pages publiques, écrans d'authentification et les cinq espaces connectés avec l'ensemble de leurs modules. Le projet compile sans erreur TypeScript et a été vérifié contre l'API réelle.

1. Direction artistique

1.1 Le point de départ

Le cahier des charges écarte explicitement le minimalisme froid façon Linear ou Notion, ainsi que les interfaces uniquement blanches et grises. Plutôt que de composer une identité générique de SaaS, l'identité est ancrée dans le contexte réel du produit : l'immobilier ouest-africain, à Cotonou, en francs CFA.

1.2 Palette

RÔLE	NOM	VALEUR	USAGE
Structure	Vert lagune	`#0E4D45`	Couleur de marque, barre latérale, boutons principaux
Structure foncée	Lagune profonde	`#08302B`	Fonds sombres, dégradés, pied de page
Accent	Laiton	`#C8963E`	Repères, chiffres, éléments de signature
Signal	Latérite	`#C25C3C`	Erreurs et actions destructrices, avec parcimonie
Texte	Encre	`#101F1C`	Corps de texte en thème clair
Fond	Sable	`#FAF7F2`	Fond de page en thème clair

Le vert lagune renvoie au littoral béninois ; le laiton au travail du bronze, tradition métallurgique du pays. La latérite — la terre rouge locale — sert uniquement de signal d'alerte, jamais de couleur décorative.

Toutes les surfaces passent par des variables CSS (`--surface` , `--line` , `--content`). Aucun composant ne code une couleur de thème en dur, ce qui rend la bascule clair/sombre entièrement déclarative.

1.3 Typographie

Trois familles, chacune avec un rôle distinct :

FAMILLE	RÔLE	JUSTIFICATION
Fraunces	Titres	Serif à optique variable, apporte le caractère patrimonial attendu sur un produit immobilier haut de gamme
Manrope	Texte courant	Grotesque humaniste, très lisible aux petites tailles, excellent support du français
JetBrains Mono	Données	Références de biens, prix au m², dates, identifiants

Le choix du monospace pour les données n'est pas cosmétique : une référence comme `BIEN-ATL-00001` **est** un code. Le composer dans la même fonte que la prose brouille la distinction entre ce qui se lit et ce qui s'identifie.

1.4 Élément de signature

Chaque carte de bien porte un encadrement dit « fiche cadastrale » : deux équerres de laiton en angles opposés, qui s'écartent au survol, et la référence du bien sur un onglet laiton — comme la languette d'un dossier foncier.

```
.cadastre::before {
  top: 10px; left: 10px;
  border-width: 1.5px 0 0 1.5px;
  border-color: rgb(200 150 62 / 0.85);
  transition: all 0.35s cubic-bezier(0.22, 1, 0.36, 1);
}
.cadastre:hover::before { top: 6px; left: 6px; width: 22px; height: 22px; }
```

C'est le seul endroit où le produit prend un risque graphique. Tout le reste — espacements, hiérarchie, boutons — reste sobre et discipliné.

1.5 Mouvement

L'animation sert la compréhension, jamais la démonstration :

- **Entrées échelonnées** sur les grilles de cartes (50 ms de décalage, plafonné à 300 ms) — l'œil suit l'ordre de lecture ;
- **Indicateur de navigation partagé** (`layoutId`) qui glisse d'un onglet à l'autre plutôt que d'apparaître brutalement ;
- **Halo pulsé** sur le bouton de l'assistant et le repère cartographique — signale un élément actif sans clignoter ;
- **Compteurs à sortie cubique** sur la page d'accueil : les chiffres se posent, ils ne défilent pas.

`prefers-reduced-motion` neutralise l'ensemble des animations.

---

2. Socle technique

DOMAINE	CHOIX
Framework	React 18 + TypeScript en mode strict
Build	Vite 6, alias `@` vers `src`, proxy `/api` vers le backend
Styles	Tailwind CSS 3, thème étendu par variables CSS
Routage	React Router 6
Données serveur	TanStack Query 5
État client	Zustand 5 avec persistance
Formulaires	React Hook Form + Zod
HTTP	Axios avec intercepteurs
Animation	Framer Motion 11
Icônes	Lucide React
Graphiques	Recharts

3. Organisation du code

```
src/
├── api/          client Axios, intercepteurs, 13 modules d'appels
├── components/
│   ├── ui/       librairie de base (24 composants)
│   ├── property/ carte, galerie, recherche, filtres
│   ├── ai/       assistant flottant, messagerie
│   ├── layout/   barre latérale, en-têtes, pied de page, navigation
│   ├── charts/   courbes, colonnes, anneau, courbe compacte
│   └── shared/   en-tête de page, carte d'indicateur
├── layouts/      gabarits public, authentification, tableau de bord
├── pages/        écrans, regroupés par espace
├── routes/       définition des routes et gardes de rôle
├── hooks/        session, requêtes, utilitaires réactifs
├── store/        session, préférences d'interface, notifications
├── types/        miroir des schémas Pydantic du backend
├── utils/        formatage, constantes métier, permissions
└── mocks/        jeu de données et serveur simulé (mode démonstration)
```

4. Trois décisions structurantes

4.1 File d'attente sur le renouvellement de jeton

Lorsqu'un tableau de bord charge six widgets simultanément et que le jeton d'accès vient d'expirer, une implémentation naïve déclenche six appels de renouvellement concurrents — dont cinq échouent, puisque le backend applique une rotation stricte des jetons de rafraîchissement.

L'intercepteur met donc les requêtes en attente pendant qu'un seul renouvellement est en cours, puis les rejoue avec le nouveau jeton :

```
if (isRefreshing) {
  return new Promise((resolve, reject) => {
    queue.push({
      resolve: (token) => {
        original.headers.Authorization = `Bearer ${token}`
        resolve(api(original))
      },
      reject,
    })
  })
}
```

Sans ce mécanisme, la rotation des jetons du backend déconnecterait l'utilisateur au premier rafraîchissement de page.

4.2 Mode démonstration

Un adaptateur Axios simulé, activé par `VITE_DEMO_MODE=true`, sert environ soixante-dix routes depuis un jeu de données réaliste : cinq agences, quatre agents, cinq propriétaires, douze biens, sept visites, trois ventes, des conversations et un journal d'audit.

Le périmètre par rôle y est reproduit à l'identique : un agent ne voit que ses biens, un client ne voit que le catalogue publié. Les fonctions d'assistance — estimation, extraction de critères, rédaction d'annonce — sont réimplémentées côté navigateur avec la même méthode que le backend.

Aucun composant, hook ou écran ne connaît l'existence de ce mode : seule la source des données change. L'interface est donc entièrement navigable sans lancer PostgreSQL, ce qui permet de la présenter depuis n'importe quel poste.

Cinq comptes de démonstration sont proposés en un clic sur l'écran de connexion :

COMPTE	RÔLE	PÉRIMÈTRE
`admin@immoflow.ai`	Super Admin	Toutes les agences
`agence@immoflow.ai`	Agence	Atlantique Immobilier
`agent@immoflow.ai`	Agent	Portefeuille personnel
`proprietaire@immoflow.ai`	Propriétaire	Biens confiés
`client@immoflow.ai`	Client	Catalogue publié

4.3 Tableaux de bord calculés, jamais illustrés

Un tableau de bord qui affiche les mêmes courbes pour tout le monde ne dit rien. Toutes les séries de la page d'accueil sont donc **agrégées à partir des données que l'API retourne à l'appelant** : aucune donnée d'exemple n'est utilisée.

```
const activitySeries = useMemo(
  () ⇒ monthlySeries(buckets, [
    { key: 'visites', rows: visits },
    { key: 'biens', rows: properties },
    { key: 'transactions', rows: [...sales, ...rentals] },
  ]),
  [buckets, visits, properties, sales, rentals],
)
```

Comme le backend filtre déjà par rôle, chaque personne obtient mécaniquement ses propres chiffres :

RÔLE	CE QUE MONTRE LA PAGE D'ACCUEIL
Super Admin	Inscriptions d'agences, répartition nationale par ville, dossiers à valider
Agence	Portefeuille par type, ventes et locations par mois, dossiers en attente
Agent	Ses biens, ses visites à confirmer, ses clients, son volume finalisé
Propriétaire	Statut de ses biens, audience de ses annonces, revenus locatifs
Client	Ses favoris, ses visites, ce que propose le catalogue

Les blocs sans objet pour un rôle ne sont pas affichés vides : ils sont absents. Un client n'a pas de section « commissions », un propriétaire pas de « dossiers à traiter ».

> **Une contrainte à connaître.** L'API plafonne `size` à 100 éléments par page. Les requêtes d'agrégation demandent donc 100 lignes au maximum ; au-delà, la validation du serveur rejette l'appel et l'écran resterait vide. C'est une erreur que nous avons commise puis corrigée : elle ne se voit qu'en testant contre l'API réelle, jamais en mode démonstration.

4.4 Écrans mutualisés, vocabulaire adapté

Les cinq espaces partagent les mêmes écrans. Un agent voit « Mes biens », une agence voit « Biens de l'agence » : c'est la même page, mais le périmètre des données et le vocabulaire diffèrent.

Ce choix évite de dupliquer quarante écrans presque identiques et garantit qu'une correction s'applique partout. La navigation par rôle est déclarée dans un seul fichier (`components/layout/navigation.ts`), ce qui rend le périmètre de chaque espace lisible d'un coup d'œil.

---

5. Parcours d'inscription

Trois portes d'entrée distinctes, parce que les trois rôles n'ont pas les mêmes prérequis :

ROUTE	PUBLIC	PARTICULARITÉ
`/inscription`	Client ou agence	Le compte agence attend une validation de la plateforme
`/inscription-proprietaire`	Propriétaire	Choix préalable de l'agence, via `GET /public/agencies`
`/accept-invitation` , `/owner/onboarding`	Agent ou propriétaire invité	Activation par jeton reçu par email

Les chemins d'activation reprennent **exactement** ceux que le backend inscrit dans ses emails. Les renommer côté frontend casserait les liens déjà envoyés : les routes françaises existent en parallèle, comme alias.

La gestion des invitations couvre le cycle complet : correction des coordonnées tant que le compte n'est pas activé, régénération d'un lien affiché et copiable, et retrait de l'équipe dont la confirmation précise ce qui est supprimé et ce qui est conservé.

---

6. Qualité de base

Ces exigences sont tenues sans être mises en avant dans l'interface :

- **Responsive** de 360 px à grand écran ; les tableaux masquent les colonnes secondaires sous 768 px plutôt que de défiler horizontalement ;
- **Focus visible** homogène sur tous les éléments interactifs ;
- **Rôles ARIA** et libellés explicites sur les dialogues, tiroirs, interrupteurs et boutons sans texte ;
- **Thème sombre** complet, sans clignotement au chargement grâce à un script inline appliqué avant le premier rendu ;
- **Chargement progressif** : squelettes calés sur les proportions réelles du contenu, jamais de saut de mise en page ;

- **États vides rédigés** : chaque écran vide explique quoi faire, il ne se contente pas d'annoncer l'absence de données.

---

7. Écriture de l'interface

Les textes suivent trois règles :

1. **Nommer ce que la personne contrôle**, pas ce que le système fait. « Retirer du catalogue », pas « Dépublier l'entité ».
2. **Un bouton dit ce qui va se passer**, et le message de confirmation reprend le même mot. « Publier » produit « Bien publié ».
3. **Une erreur explique et oriente**. « Publication impossible : au moins une photo est requise » plutôt que « Erreur 409 ».

Les messages d'erreur de l'API sont traduits par une fonction unique (`errorMessage`) qui extrait le champ manquant lorsque le backend le fournit, de sorte que l'utilisateur lit une phrase actionnable et non un code technique.

PARTIE 3

Installation

Prérequis, build de production, configuration Nginx et dépannage.

Prérequis

OUTIL	VERSION MINIMALE	VÉRIFIER
Node.js	18 (20 ou 22 recommandé)	`node -v`
npm	9	`npm -v`

Un navigateur récent est nécessaire (Chrome, Firefox, Edge ou Safari à jour).

Installation

```
unzip immoflow-ai-frontend.zip
cd immoflow-ai-frontend
npm install
```

L'installation prend une trentaine de secondes et récupère environ 215 paquets.

Le fichier `.env` est déjà présent dans l'archive. Si vous partez d'un dépôt Git, créez-le à partir du modèle :

```
cp .env.example .env
```

Lancement

```
npm run dev
```

```
VITE v6.4.3  ready in 408 ms
➜  Local:   http://localhost:5173/
```

Brancher le backend

Le frontend attend l'API sur `http://localhost:8000`.

```
cd ../immoflow-ai-api
cp backend/.env.example backend/.env    # renseignez SECRET_KEY
docker compose up -d
curl http://localhost:8000/health       # doit répondre "healthy"
```

Vite relaie `/api/*` vers cette adresse. Pour cibler un autre hôte :

```
VITE_PROXY_TARGET=http://192.168.1.20:8000
```

Redémarrez `npm run dev` après toute modification de `.env` : les variables sont lues au démarrage du serveur, pas à chaud.

Build de production

```
npm run build      # génère dist/
npm run preview    # sert dist/ sur http://localhost:4173
```

Le dossier `dist/` est un site statique : déposez-le sur Netlify, Vercel, S3, Nginx ou tout hébergeur de fichiers.

Configuration Nginx

L'application utilise le routage par historique : toutes les routes doivent renvoyer `index.html`, sinon un rafraîchissement sur `/agence/biens` produit une erreur 404.

```
server {
    listen 80;
    root /var/www/immoflow;
    index index.html;

    location / {
        try_files $uri $uri/ /index.html;
    }

    location /api/ {
        proxy_pass http://127.0.0.1:8000;
        proxy_set_header Host $host;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    }
}
```

En production, `VITE_API_URL` doit pointer vers l'URL publique de l'API si celle-ci n'est pas servie derrière le même domaine.

Dépannage

« L'API est injoignable » sur chaque écran Le backend n'est pas démarré, ou n'écoute pas sur le port attendu. Vérifiez avec `curl http://localhost:8000/health`. Pour travailler sans backend, passez `VITE_DEMO_MODE=true` dans `.env` et redémarrez.

Le port 5173 est déjà utilisé

```
npm run dev -- --port 5174
```

Les polices ne s'affichent pas Fraunces, Manrope et JetBrains Mono sont chargées depuis Google Fonts. Sans connexion Internet, le navigateur retombe sur les polices système : la mise en page reste correcte, seul le caractère typographique change.

Les photos des biens de démonstration ne s'affichent pas Elles proviennent d'Unsplash et nécessitent une connexion. Un dégradé de marque prend le relais, la mise en page ne casse pas.

Écran blanc après `npm run build` Vérifiez que le serveur renvoie bien `index.html` sur toutes les routes (voir la configuration Nginx ci-dessus).

Erreurs TypeScript après modification

```
npm run typecheck
```

Le projet est livré avec zéro erreur : toute erreur provient d'une modification locale.

PARTIE 4

Variables d'environnement

Chaque variable, son effet et les configurations types.

Toutes les variables sont préfixées `VITE_` : c'est la condition pour que Vite les expose au navigateur. Elles sont lues **au démarrage** du serveur — après modification de `.env`, relancez `npm run dev`.

> Ces valeurs se retrouvent dans le bundle envoyé au navigateur. **N'y placez jamais de secret** : pas de clé d'API privée, pas de mot de passe, pas de jeton de service.

Référence

`VITE_API_URL`

Préfixe de tous les appels HTTP.

- Défaut : `/api/v1`
- En développement, laissez cette valeur : le proxy Vite se charge du relais.
- En production sur un domaine distinct : `https://api.votre-domaine.com/api/v1`

`VITE_PROXY_TARGET`

Adresse du backend visée par le proxy de développement. **Sans effet en production** : le proxy n'existe que dans le serveur Vite.

- Défaut : `http://localhost:8000`

`VITE_WS_URL`

Adresse du WebSocket temps réel, utilisée par la messagerie.

- Défaut : `ws://localhost:8000/ws`
- En production derrière HTTPS : `wss://api.votre-domaine.com/ws`

`VITE_DEMO_MODE`

Bascule entre l'API réelle et le serveur simulé.

VALEUR	COMPORTEMENT
`false` (défaut)	Les requêtes partent vers `VITE_API_URL`
`true`	L'adaptateur HTTP est remplacé : aucune requête ne sort du navigateur

En mode démonstration, l'écran de connexion propose cinq comptes correspondant aux cinq rôles. C'est le mode à utiliser pour présenter l'interface sans infrastructure.

À laisser sur `false` en production.

`VITE_APP_NAME`

Nom affiché dans l'interface.

- Défaut : `ImmoFlow AI`

Fichiers d'exemple

Développement avec le backend local

```
VITE_API_URL=/api/v1
VITE_PROXY_TARGET=http://localhost:8000
VITE_WS_URL=ws://localhost:8000/ws
VITE_DEMO_MODE=false
VITE_APP_NAME=ImmoFlow AI
```

Démonstration hors ligne

```
VITE_API_URL=/api/v1
VITE_DEMO_MODE=true
VITE_APP_NAME=ImmoFlow AI
```

Production

```
VITE_API_URL=https://api.votre-domaine.com/api/v1
VITE_WS_URL=wss://api.votre-domaine.com/ws
VITE_DEMO_MODE=false
VITE_APP_NAME=ImmoFlow AI
```

PARTIE 5

Guide d'utilisation

Le site public, puis chacun des cinq espaces connectés.

Ce guide suit le produit dans l'ordre où on le découvre : le site public, puis chacun des cinq espaces connectés.

1. Site public

Accueil (`/`)

Le bandeau d'ouverture propose deux façons de chercher, sélectionnables par un interrupteur :

- **Par critères** — mot-clé, ville, type d'opération, budget maximum ;
- **Recherche intelligente** — une phrase libre, par exemple « maison 4 chambres proche centre budget 60 millions ». La phrase est traduite en critères, puis appliquée au catalogue.

Plus bas : les dernières publications, les quatre capacités de la plateforme, le parcours des quatre rôles et les témoignages.

Catalogue (`/biens`)

Filtres à gauche sur grand écran, dans un tiroir sur mobile : opération, type de bien, ville, quartier, budget, nombre de chambres, surface, équipements. Le tri se fait par date, prix, surface ou nombre de consultations.

Quand la recherche intelligente est utilisée, la réponse de l'assistant s'affiche au-dessus des résultats et les filtres sont pré-remplis — vous pouvez les ajuster ensuite à la main.

Fiche d'un bien (`/biens/{id}`)

Galerie en mosaïque, ouvrable en plein écran (flèches et touche `Échap` au clavier), prix avec le prix au m² calculé, caractéristiques, description, équipements, repère de localisation et biens similaires dans la même ville.

Deux actions : **demander une visite** et **contacter l'agent**. Les deux exigent d'être connecté ; un visiteur non connecté est redirigé vers la connexion, puis ramené sur la fiche.

---

2. Créer un compte

Client ou agence — `/inscription`

- **Je cherche un bien** → compte client, actif immédiatement ;
- **Je dirige une agence** → compte agence, en attente de validation par la plateforme.

Propriétaire — `/inscription-proprietaire`

Le propriétaire suit un parcours distinct, en deux étapes, parce qu'un bien se confie toujours à une agence précise :

1. **Choisir l'agence** parmi celles qui sont actives sur la plateforme ;
2. **Créer son compte** — il peut se connecter immédiatement.

Son dossier est alors en attente de vérification : il dépose sa pièce d'identité depuis l'espace Documents, puis l'agence valide. Sans pièce d'identité, la validation est refusée par l'API — ce n'est pas un contrôle d'affichage.

Agent ou propriétaire invité — lien reçu par email

Quand l'agence prend l'initiative, elle crée la fiche et un lien d'activation part par email (`/accept-invitation` pour un agent, `/owner/onboarding` pour un propriétaire). Il suffit de choisir un mot de passe.

Le lien est valable 48 heures. **Régénérer un lien invalide tous les précédents** : si vous en avez généré un nouveau, l'ancien affichera « invitation expirée », ce qui est le comportement voulu.

Le rôle **agent** ne s'obtient jamais par inscription libre.

---

3. Espace Super Admin (`/admin`)

Tableau de bord — agences actives, demandes en attente, utilisateurs, biens, courbes de croissance et répartition par ville.

Agences — le cœur du rôle. Chaque nouvelle inscription arrive en « À valider ». Ouvrez le dossier pour vérifier le RCCM et l'identifiant fiscal, puis validez ou rejetez. Un rejet demande un motif, transmis au dirigeant.

Une agence validée devient opérationnelle : son dirigeant peut se connecter et inviter son équipe. Une agence suspendue bloque immédiatement la connexion de tous ses agents.

Utilisateurs — tous les comptes, tous rôles confondus, avec leur dernière connexion.

Journal d'audit — chaque action sensible, son auteur, son résultat, l'adresse IP et l'identifiant de requête. Filtrable par type de ressource. Ces enregistrements ne sont ni modifiables ni supprimables.

---

4. Espace Agence (`/agence`)

Agents — « Inviter un agent » crée le compte et envoie un lien d'activation valable 48 heures. Le lien est aussi affiché à l'écran, avec un bouton de copie : si votre serveur SMTP n'est pas configuré, vous pouvez le transmettre autrement.

Tant que l'agent n'a pas activé son compte, le menu de sa ligne propose :

- **Corriger l'invitation** — nom, adresse email, téléphone, fonction. Indispensable en cas de faute de frappe dans l'adresse, qui rendrait le lien inutilisable ;
- **Générer un nouveau lien** — affiché et copiable, il remplace les précédents.

Retirer de l'équipe distingue deux cas, précisés dans la confirmation : un agent jamais activé est supprimé entièrement et son adresse redevient disponible ; un agent qui a été actif perd tout accès, mais son compte est conservé désactivé pour que les biens et transactions qu'il a traités gardent leur auteur. Vous pourrez le réintégrer plus tard avec la même adresse.

Deux droits se règlent à l'invitation et se modifient ensuite :

DROIT	EFFET
Gérer les propriétaires	Inviter des propriétaires et suivre leurs dossiers
Publier des biens	Mettre une annonce en ligne sur le catalogue public

Propriétaires — invitez un propriétaire, il complète son dossier et dépose sa pièce d'identité. Vous validez ensuite.

> La validation est **refusée par l'API** tant qu'aucune pièce d'identité n'a été déposée. Ce n'est pas un contrôle d'affichage : la conformité documentaire est garantie côté serveur.

Biens — deux vues : tableau pour piloter, galerie pour vérifier le rendu public. Un bien créé démarre en brouillon. Sa publication exige un titre, un prix, une ville, une description d'au moins 40 caractères et au moins une photo. À défaut, l'interface affiche précisément ce qui manque.

Clients — fiches avec budget, origine du contact et notes. Un client créé par un agent lui est automatiquement rattaché.

Visites — les demandes arrivent en « Demandée ». Vous confirmez, proposez un autre créneau ou refusez. Après la visite, clôturez avec une note d'intérêt de 1 à 5 et un compte rendu.

Transactions — ventes et locations. Le statut de la transaction pilote celui du bien : une vente créée réserve le bien, une vente finalisée le passe en vendu et le retire du catalogue. Un bail activé passe le bien en loué.

Documents — pièces d'identité, titres de propriété, mandats, contrats. Chaque pièce est validée ou rejetée avec motif. Le téléchargement passe par l'API, qui contrôle les droits et journalise l'accès.

Messagerie — conversations avec clients, propriétaires et collègues, toujours cloisonnées à votre agence.

Assistant IA — voir la section 8.

---

5. Espace Agent (`/agent`)

Mêmes écrans que l'agence, mais restreints à ce qui vous est attribué : vos biens, vos propriétaires, vos clients, vos visites, vos transactions.

Cette restriction est appliquée par le serveur, pas par l'affichage : une adresse saisie à la main ne donne pas accès au portefeuille d'un collègue.

Les ventes et les locations sont séparées en deux entrées de menu, mais ouvrent la même page sur l'onglet correspondant.

---

6. Espace Propriétaire (`/proprietaire`)

Consultation seule, ce qui est voulu : l'agence gère, vous suivez.

- **Mes biens** — état, statut, prix et consultations de chaque bien confié ;
- **Visites** — les rendez-vous programmés sur vos biens ;
- **Transactions** — les ventes et locations vous concernant ;
- **Documents** — vos pièces et leur statut de vérification ;
- **Messages** — échanges avec votre agent référent.

---

7. Espace Client (`/espace-client`)

- **Rechercher** — le catalogue, avec vos critères ;
- **Favoris** — les biens mis de côté, retirables d'un clic ;
- **Historique** — vos recherches passées, relançables ;
- **Mes demandes** — vos demandes de renseignements et leur statut ;
- **Mes visites** — vos rendez-vous, demandés ou confirmés ;
- **Messages** — échanges avec les agents.

---

8. Assistant immobilier

Le bouton flottant est présent sur tous les écrans connectés. Raccourci : `Ctrl` + `K` (`Cmd` + `K` sur Mac).

Rechercher

Décrivez le bien en une phrase. L'assistant extrait la ville, le quartier, le type, le nombre de chambres, le budget et les équipements, puis propose les biens correspondants **dans votre périmètre** : un client ne voit que le catalogue publié.

Estimer

Donnez le type, la surface et le quartier. Le prix conseillé est calculé à partir de la **médiane du prix au m² des biens comparables réellement enregistrés**. Si moins de trois comparables existent, un barème régional prend le relais.

Le résultat affiche systématiquement le nombre de comparables, la méthode employée et un indice de confiance. Un prix sans sa méthode ne se défend pas devant un propriétaire.

Interroger le marché

L'assistant distingue une recherche d'une question de marché. « Quel est le prix moyen à Cotonou ? » ne renvoie pas une liste de biens mais une médiane, une fourchette et un prix au m², calculés sur les annonces de votre périmètre.

Les ventes et les locations sont calculées séparément : une médiane qui mêlerait un loyer mensuel et un prix d'acquisition n'aurait aucun sens.

Rédiger

À partir des caractéristiques, l'assistant propose un titre et une description en français, modifiables avant publication.

Atelier IA

La page dédiée (`/agence/assistant` , `/agent/assistant`) ajoute le **contrôle qualité** d'une annonce publiée : score sur 100, champs manquants, points de vigilance rédactionnels et recommandations.

---

9. Préférences

`/profil` regroupe le profil, la fiche de l'agence, le changement de mot de passe et le choix du thème clair ou sombre. Le changement de mot de passe ferme toutes vos autres sessions.

---

10. En cas de problème

« L'API est injoignable » — le backend n'est pas démarré. Lancez-le, ou passez en mode démonstration (`VITE_DEMO_MODE=true`).

« Session expirée » — le renouvellement automatique du jeton a échoué. Reconnectez-vous.

« Publication impossible » — le message liste exactement les éléments manquants.

Une action est absente d'un menu — votre rôle ne l'autorise pas. Un agent sans le droit de publication ne voit pas l'action « Publier ».